

FOMOKiller

Autor

Anthony Gómez

Tutor

Tomas Escudero

Proyecto Intermodular DAW

Curso 2025 - 2026

Centro Educativo: IFP Julián Camarillo

Convocatoria de Presentación: Abril 2026

Contenido

Introducción.....	3
Justificación del proyecto	¡Error! Marcador no definido.
Objetivo General.....	4
Estado del arte.....	5
Metodología de trabajo	6
Planificación	7
Resultados	8
Fases	8
Definición:	8
Gestor:	8
Administrador:.....	9
Diseño.....	10-13
Desarrollo	13-14
Conclusiones:	15
Bibliografía.....	16

Introducción

El objetivo de este proyecto es el desarrollo de una aplicación web que permita a los usuarios gestionar su backlog de videojuegos de una forma dinámica, visual e intuitiva. Con FOMOKiller se busca eliminar el síndrome FOMO (Fear Of Missing Out) que experimentan los jugadores ante la constante acumulación de títulos pendientes, ofreciendo una plataforma que prioriza la experiencia de descubrimiento y la organización personal.

La aplicación combina un sistema de swipe al estilo de las plataformas de descubrimiento modernas con una gestión completa del backlog y un catálogo de colecciones curadas. De este modo, el usuario no solo lleva un registro de los juegos que quiere jugar, sino que también descubre nuevos títulos de forma gamificada y sin fricciones.

Para llevar a cabo este proyecto se han analizado las necesidades reales de los jugadores actuales, quienes se enfrentan a un mercado saturado de lanzamientos y tienen dificultades para decidir qué jugar a continuación. A partir de esa investigación se han definido los requisitos funcionales, se ha planificado el trabajo por semanas y se ha desarrollado una solución full-stack que integra una API propia, una base de datos relacional y un cliente web moderno.

Justificación del proyecto

FOMOKiller surge de una problemática muy reconocible en la comunidad gamer: la acumulación incontrolada de videojuegos sin jugar y la parálisis por elección que provoca. Plataformas como Steam, la PlayStation Store o Xbox Game Pass facilitan la adquisición masiva de juegos, pero ninguna ofrece al usuario una herramienta sencilla para gestionar qué tiene pendiente, qué le interesa realmente y qué ha descartado.

Existen aplicaciones como Backloggd o HowLongToBeat que cubren parcialmente esta necesidad, pero su enfoque es principalmente estadístico o de reseña social, no de ayuda activa en la toma de decisiones. FOMOKiller propone un enfoque diferente: un mecanismo de swipe que permite al usuario calificar juegos rápidamente (me interesa / no me interesa), construyendo así un backlog personalizado de forma natural y sin esfuerzo.

Además, la plataforma incorpora colecciones temáticas curadas por administradores, una sección de exploración por géneros y plataformas, y un perfil personalizable. Todo ello con un diseño visual inmersivo inspirado en la estética retro de ps2 (conocida como EmotionStation), que hace de la propia aplicación una experiencia disfrutable.

Objetivo General

Desarrollar una aplicación web full-stack que permita a los usuarios gestionar su backlog de videojuegos mediante un sistema de swipe interactivo, ofreciendo además funcionalidades de exploración por catálogo, colecciones curadas y un panel de administración completo para la gestión de contenido y usuarios.

Estado del arte

Para contextualizar el proyecto se llevó a cabo una investigación sobre las soluciones existentes en el mercado orientadas a la gestión de videojuegos y al descubrimiento de nuevo contenido.

Backloggd es actualmente la plataforma más popular para el registro de videojuegos jugados. Permite llevar listas de jugados, en progreso, en backlog y abandonados, así como escribir reseñas y seguir a otros usuarios. Sin embargo, no ofrece ningún mecanismo activo de ayuda para decidir qué jugar a continuación, y su interfaz está orientada principalmente al registro retrospectivo.

HowLongToBeat se enfoca en informar al usuario del tiempo estimado de finalización de cada juego, lo cual es útil para planificar, pero no dispone de un sistema de gestión de backlog ni de descubrimiento de títulos nuevos.

Steam cuenta con una lista de deseos y registros de biblioteca, pero estas funciones son incidentales dentro de una plataforma de venta. No existe ninguna herramienta de gestión activa ni de descubrimiento gamificado.

RAWG.io ofrece una base de datos masiva de videojuegos con etiquetas, géneros y plataformas, y dispone de una API pública muy completa. Sin embargo, es principalmente un catálogo de consulta, no una herramienta de gestión personal.

Ninguna de estas plataformas combina la gestión de backlog con un mecanismo de decisión ágil. Es precisamente en ese hueco donde FOMOKiller aporta valor diferencial: un flujo de swipe que convierte una tarea tediosa — revisar el backlog y decidir— en una experiencia rápida e incluso entretenida.

También se detectó que los usuarios más jóvenes están familiarizados con las mecánicas de swipe (Tinder, TikTok) y responden bien a interfaces que requieren decisiones binarias y rápidas. Adaptar esta interacción al contexto de los videojuegos resultó ser una propuesta con gran potencial de usabilidad.

Metodología de trabajo

Para la organización del proyecto se ha utilizado una metodología iterativa basada en sprints semanales, complementada con un tablero de tareas dividido en tres estados: pendiente, en progreso y completado. Esto ha permitido mantener visibilidad sobre el estado del desarrollo en todo momento y adaptarse a los cambios de requisitos o a los problemas técnicos que fueron surgiendo.

El proyecto se dividió en dos grandes bloques de trabajo en paralelo: el desarrollo del servidor (Back End) y el desarrollo del cliente (Front End). Ambas partes se integraron progresivamente conforme se completaban funcionalidades, comprobando en cada sprint que la comunicación entre cliente y servidor funcionaba correctamente.

Durante el desarrollo se presentaron varios retos técnicos, como la integración con la API de RAWG para la importación de juegos, el diseño del sistema de autenticación basado en cookies HttpOnly, la implementación del mecanismo de swipe con animaciones fluidas en el cliente, y el diseño final de la web. En cada caso se investigó la solución más adecuada, se prototipó y se integró en el flujo principal.

Planificación

Semana	Fechas	Tarea	Descripción
Semana 1	03/03/25 – 09/03/25	Investigación y requisitos	Análisis de plataformas similares, definición de funcionalidades clave y modelado inicial de base de datos.
Semana 2	10/03/25 – 16/03/25	Diseño de la arquitectura	Definición del stack tecnológico, estructura de carpetas, diseño del esquema de base de datos y primeros endpoints de la API.
Semana 3	17/03/25 – 23/03/25	Back End: Auth y usuarios	Implementación del sistema de registro, login y autenticación con cookies HttpOnly. Middleware de roles.
Semana 4	24/03/25 – 30/03/25	Back End: Juegos y RAWG	Modelos de Game, Genre, Platform. Integración con la API de RAWG para importación de juegos al catálogo.
Semana 5	31/03/25 – 06/04/25	Back End: Backlog y Colecciones	Endpoints de gestión del backlog (swipe), colecciones, y lógica de recomendaciones.
Semana 6	07/04/25 – 13/04/25	Front End: Landing y Auth	Desarrollo de la landing page, páginas de login y registro. Layout de navegación.
Semana 7	14/04/25 – 20/04/25	Front End: Swipe y Backlog	Implementación del mecanismo de swipe con animaciones, vista de backlog con filtros y paginación.
Semana 8	21/04/25 – 27/04/25	Front End: Explorar y Perfil	Página de exploración con filtros por género y plataforma. Perfil de usuario editable.
Semana 9	28/04/25 – 04/05/25	Panel de Administración	CRUD completo de usuarios, juegos y colecciones desde el panel admin.
Semana 10	05/05/25 – 11/05/25	Pulido visual y responsive	Ajustes de diseño, mejoras de accesibilidad por teclado, adaptación móvil y resolución de bugs.

Resultados

Fases

Para alcanzar los objetivos del proyecto se estructuró el desarrollo en las siguientes fases, cada una con entregables y criterios de aceptación definidos.

Definición:

En esta fase se establecieron los requisitos que la aplicación debía satisfacer para cada tipo de actor: el usuario estándar (gestor) y el administrador. También se definió el modelo de datos relacional y la arquitectura general de la solución.

Se identificaron las siguientes entidades principales: User, Game, Genre, Platform, Collection, CollectionGame, UserGame (backlog). Las relaciones entre ellas reflejan que un usuario tiene múltiples entradas en su backlog, los juegos pertenecen a géneros y plataformas, y las colecciones agrupan juegos seleccionados por el administrador.

Gestor:

El usuario estándar dispone de las siguientes funcionalidades:

- **Swipe de descubrimiento:** el usuario visualiza tarjetas de juegos y puede marcarlos como "me interesa" (los añade al backlog) o "no me interesa" (los descarta). El gesto puede realizarse con ratón y teclado (flechas izquierda/derecha).
- **Backlog personal:** lista de juegos guardados con filtros por estado (pendiente, completado, Top5), género y plataforma. Incluye paginación y búsqueda por título. Además de poder curar su propio backlog mediante un sistema de swipe integrado.
- **Exploración de catálogo:** Vista donde se detalla la lista de prioridades de la persona, en la que puede reorganizar un máximo de 5 juegos elegidos desde el backlog,
- **Colecciones:** acceso a las colecciones temáticas creadas por los administradores, con posibilidad de explorar los juegos de cada una. Además de un buscador (por el nombre del título) para añadir juegos de forma más directa al backlog.
- **Perfil:** visualización y edición de los datos personales (nombre de usuario, bio, avatar), así como estadísticas básicas de actividad. Incluye la posibilidad de reiniciar el mazo de swipe y de volver a hacer el cuestionario por si sus gustos cambian en algún momento.

Administrador:

El usuario con rol administrador, además de todas las funcionalidades del gestor, dispone de acceso al panel de administración con las siguientes capacidades:

- **Gestión de usuarios:** listado completo con búsqueda por nombre o email, creación de nuevos usuarios, edición de datos (incluyendo cambio de contraseña y rol), y eliminación.
- **Gestión de juegos:** listado con filtro por título, creación manual de juegos con todos sus campos, importación desde la API de RAWG buscando por nombre.
- **Gestión de colecciones:** creación y edición de colecciones con título, descripción e imagen. Puede además añadir o quitar juegos de cada colección con búsqueda en tiempo real.

Diseño

El diseño de FOMOKiller se trabajó desde los wireframes iniciales hasta la identidad visual final, tomando como referencia la estética retro-tecnológica de los videojuegos clásicos.

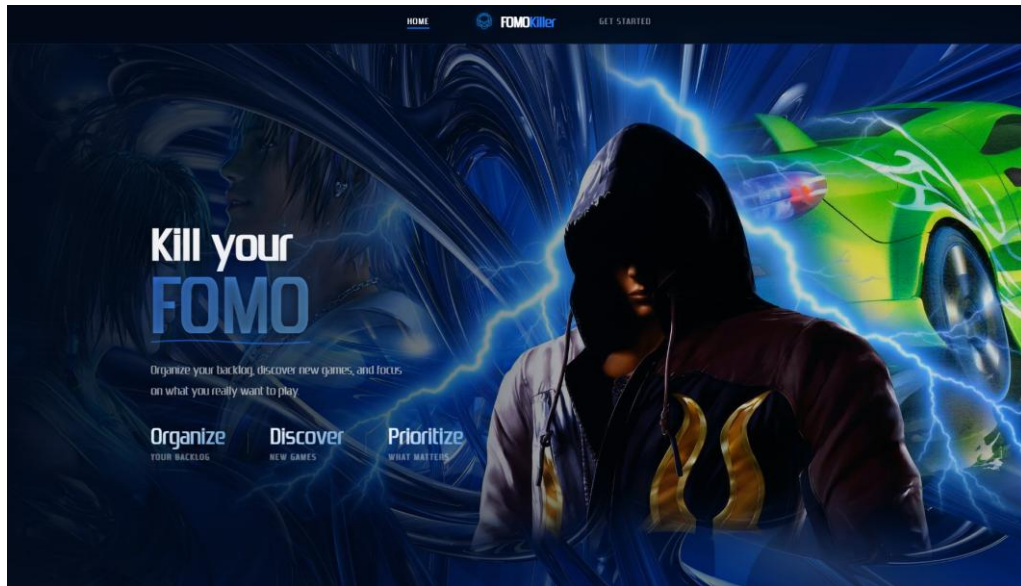
Sistema de color: fondo oscuro profundo (#030D1F), azul cobalto como color de acento primario (#0047AB y su variante brillante #2979FF), verde para acciones positivas (#00C853) y rojo para acciones de descarte (#F44336). Este sistema de color transmite inmersión y familiaridad para el público objetivo.

Tipografía: se combinan tres familias tipográficas: Zrnic (fuente display personalizada para títulos y elementos de marca), Outfit (fuente principal de texto por su legibilidad en pantalla) y Oxanium (para badges, etiquetas y elementos técnicos secundarios).

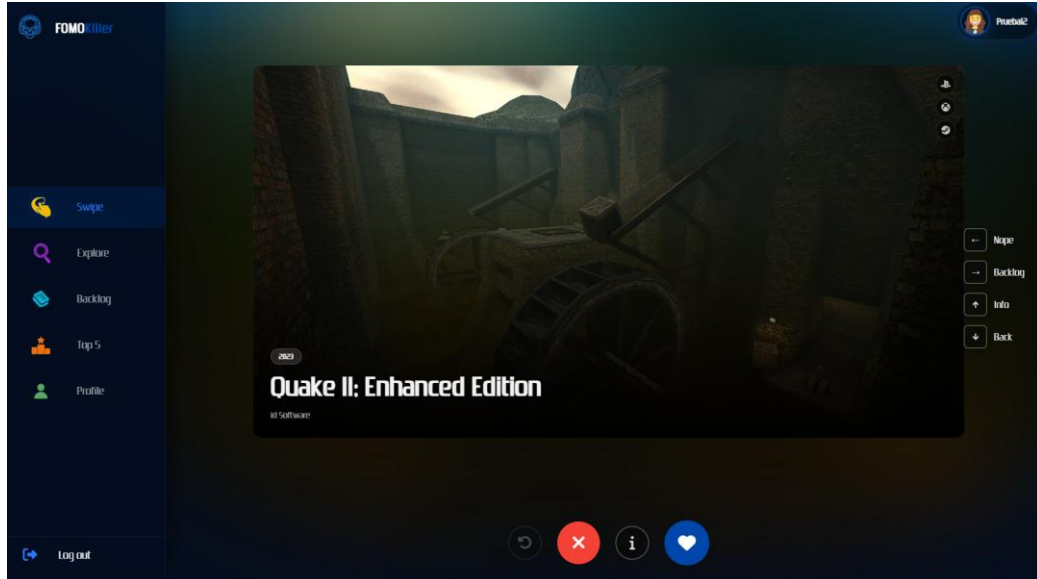
Componentes visuales: la interfaz incorpora efectos de scanlines y viñeta en el fondo, bordes con glow en elementos interactivos, y transiciones suaves. Las tarjetas de swipe tienen animación física con rotación proporcional al desplazamiento. Los modales cierran con clic exterior, tecla Escape o barra espaciadora para agilizar la navegación por teclado.

A continuación se describen las principales vistas:

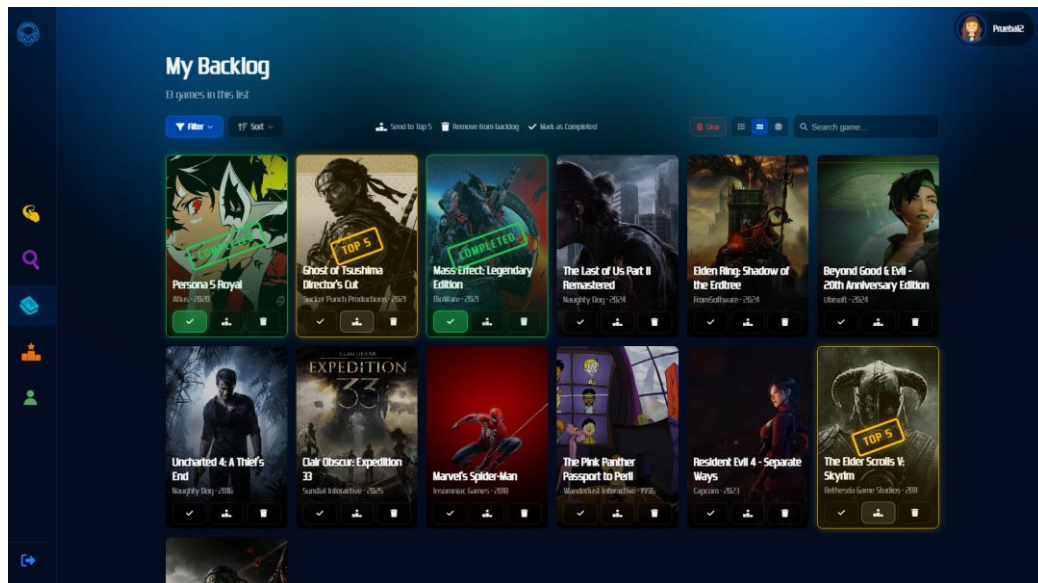
- **Landing page:** página de presentación con hero animado, sección de características, vista previa de la mecánica de swipe y llamada a la acción.



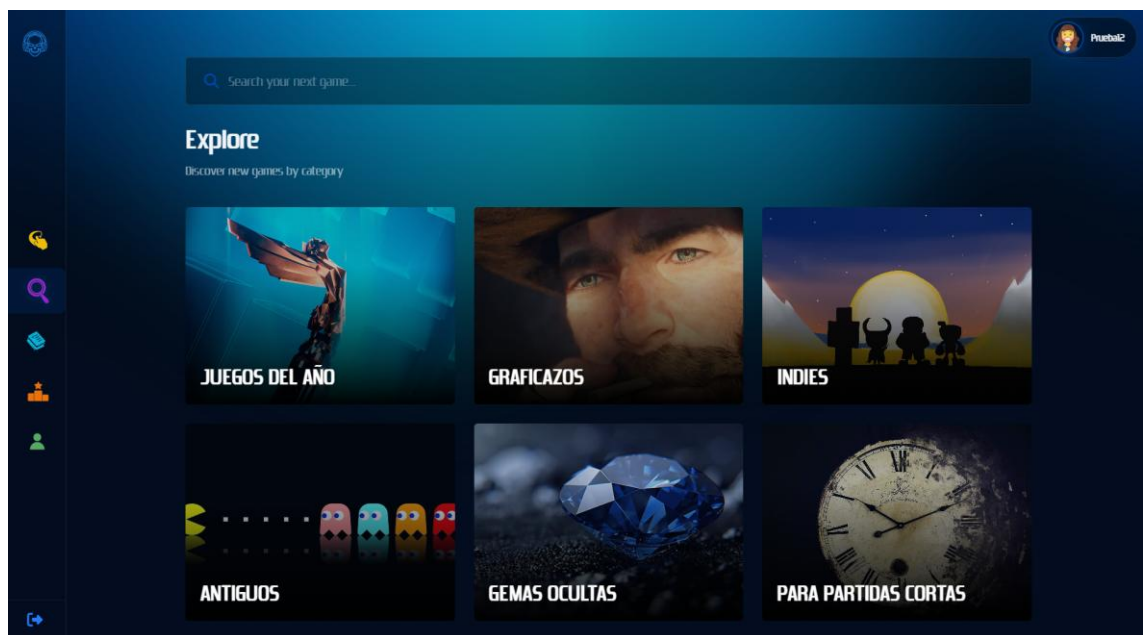
- **Swipe:** tarjetas de juegos apiladas visualmente con información básica (portada, título, géneros, plataformas). Indicadores visuales de like y pass al arrastrar.



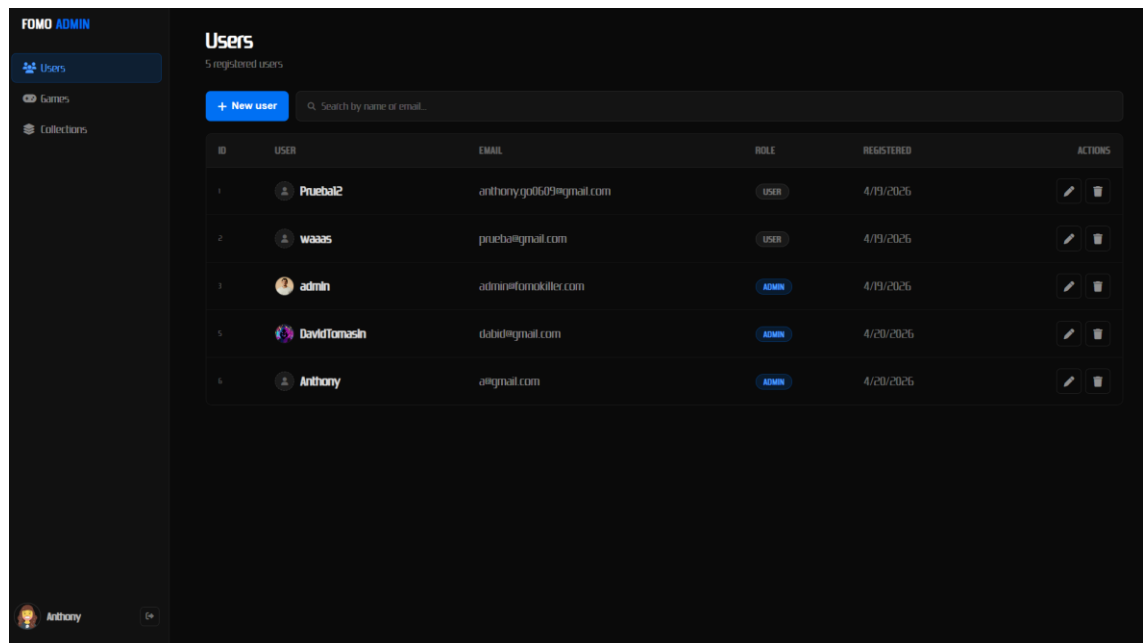
- **Backlog:** cuadrícula de tarjetas con imagen de portada, título y estado, con panel de filtros lateral.



- **Explorar:** cuadrícula paginada con filtros superiores.



- **Admin:** layout de panel con sidebar de navegación fijo y área de contenido con tablas CRUD.



Desarrollo

Para la implementación de la aplicación se utilizaron las siguientes tecnologías:

Front End

- **React 19** con TypeScript como framework principal de interfaz de usuario.
- **Vite 7** como bundler y entorno de desarrollo, por su rapidez y soporte nativo de ESM modules.
- **React Router DOM v7** para la gestión de rutas y layouts anidados.
- **React Bits** para animaciones.
- **CSS puro** con variables CSS personalizadas para el sistema de diseño.

Back End

- **Node.js** con TypeScript como entorno de ejecución y lenguaje del servidor.
- **Express** como framework HTTP para la construcción de la API REST.
- **Sequelize** como ORM para la gestión del modelo de datos y las consultas a la base de datos.
- **MySQL** como sistema gestor de base de datos relacional, por su robustez para relaciones complejas (muchos a muchos entre juegos, géneros y plataformas).

- **RAWG Video Games Database API** para la importación de datos de juegos (portadas, géneros, plataformas, desarrolladores, año de lanzamiento).
- **Cookies HttpOnly** para proteger el token de sesión frente a ataques XSS, con middleware de roles para diferenciar rutas de usuario y administrador.
- **Seguridad:** JWT (JSON Web Tokens) y Bcrypt.js para cifrado de contraseñas.

Infraestructura:

- **Docker & Docker Compose:** Orquestación de contenedores para Base de Datos, Backend y Frontend.

El servidor sigue una arquitectura en capas con separación entre rutas, controladores y modelos. El cliente organiza las vistas en tres layouts: LandingLayout (páginas públicas), AppLayout (usuario autenticado con navbar) y AdminLayout (panel de administración con sidebar).

Conclusiones

Tras el desarrollo de FOMOKiller se han alcanzado todos los objetivos planteados al inicio del proyecto. La aplicación es funcional en su totalidad, con un flujo de usuario completo desde el registro hasta la gestión del backlog, y un panel de administración operativo para el mantenimiento del contenido.

La implementación del mecanismo de swipe resultó ser uno de los retos más interesantes del proyecto, tanto a nivel de interfaz como a nivel de lógica por la necesidad de sincronizar el estado local con el servidor de forma eficiente.

La integración con la API de RAWG supuso un aprendizaje valioso sobre el consumo de APIs externas y la transformación de datos externos al modelo de datos propio, incluyendo la gestión de relaciones muchos a muchos.

En cuanto a las mejoras a futuro, se contemplan las siguientes líneas de desarrollo:

- **Modo social:** Posibilidad de seguir a otros usuarios, ver sus backlogs públicos y compartir colecciones.
- **Aplicación móvil nativa:** Una app nativa mejoraría significativamente la experiencia en dispositivos móviles.
- **Integración con plataformas de juego:** conexión con las APIs de Steam o PlayStation para importar automáticamente la biblioteca del usuario.
- **Chatbot con IA:** Para desbloquear una nueva forma de recomendación de videojuegos al implementar nuestro propio agente de IA.

En definitiva, FOMOKiller demuestra que es posible abordar un problema cotidiano del mundo gamer con una solución técnica sólida y una experiencia de usuario cuidada, combinando tecnologías modernas de desarrollo web con un diseño visual distintivo.

Bibliografía

Express.js. (2024). *Express – Fast, unopinionated, minimalist web framework for Node.js*. <https://expressjs.com>

Facebook Open Source. (2024). *React – A JavaScript library for building user interfaces*. <https://react.dev>

Google Fonts. (2024). *Outfit & Oxanium typefaces*. <https://fonts.google.com>

MDN Web Docs. (2024). *CSS Custom Properties (variables)*. Mozilla. https://developer.mozilla.org/es/docs/Web/CSS/Using_CSS_custom_properties

MySQL. (2024). *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/>

OWASP. (2024). *HttpOnly Cookie – OWASP Cheat Sheet Series*. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

PCGamesN. (19 de junio de 2024). *Steam users have spent \$19 billion on games they've never even played*. <https://www.pcgamesn.com/steam/pile-of-shame>

RAWG Video Games Database. (2024). *RAWG API Documentation*. <https://rawg.io/apidocs>

React Router. (2024). *React Router v7 Documentation*. <https://reactrouter.com>

Sequelize. (2024). *Sequelize ORM – Node.js ORM for Oracle, Postgres, MySQL, MariaDB, SQLite and SQL Server*. <https://sequelize.org>

Vite. (2024). *Vite – Next Generation Frontend Tooling*. <https://vitejs.dev>